	Pimpri-Chinchwad Education Trust's **Pimpri-Chinchwad College of Engineering** (An Autonomous Institute) Affiliated to Savitribai Phule Pune University (SPPU) ISO 21001:2018 Certified by TUV
ASSIGNMENT - 4 Dijkstra's Algorithm	

Name: Harshit Pandita **PRN:**124B1F017
Subject: Design and Analysis of Algorithms **Batch:** A1

1. Aim

Scenario: Smart Traffic Management for Emergency Vehicles

A smart city is implementing an intelligent traffic management system to assist ambulances in reaching hospitals as quickly as possible. The city's road network is represented as a graph, where:

● Intersections (junctions) are nodes.

● Roads between intersections are edges with weights representing travel time

(in minutes) considering traffic congestion.

An ambulance is currently at Source (S) and needs to reach the nearest hospital (Destination D) in the shortest

possible time. Due to dynamic traffic conditions, the weight of each road segment may change in real time.

As a transportation engineer, you are assigned to:

1. Implement Dijkstra's algorithm to find the shortest path from the ambulance's current location (S) to all possible hospitals.

2. Account for dynamic weight updates as traffic conditions change.

3. Optimize the system to work efficiently for a large city with thousands of intersections and roads.

4. Provide a visual representation of the optimal path for navigation.

THEORY – DIJKSTRA'S ALGORITHM

Dijkstra's Algorithm is a popular **Greedy algorithm** used to find the shortest path from a source vertex to all other vertices in a weighted graph. It is widely used in navigation and transportation systems to determine the minimum travel time between locations. In a smart traffic management

system, intersections are represented as vertices and roads are represented as edges, with the weight of each edge representing the travel time required to travel through that road.

The objective is to find the **shortest travel time from an ambulance's current location to all possible hospitals** and identify the hospital that can be reached in the minimum time.

Working of Dijkstra's Algorithm

Dijkstra's Algorithm works through the following steps:

1. Initialization:

The distance of the source vertex is initialized to **0**, while the distance of every other vertex is set to infinity. A priority queue is used to efficiently select the vertex having the smallest known distance.

2. Select the Minimum-Distance Vertex:

The unvisited vertex with the smallest tentative distance is selected as the current vertex.

3. Relaxation:

For each neighboring vertex, the algorithm checks whether travelling through the current vertex provides a shorter path. If a shorter path is found, the distance is updated.

The relaxation condition is:

If dist[v] > dist[u] + weight(u,v), then

dist[v] = dist[u] + weight(u,v)

4. Repeat:

The process continues until all reachable vertices have been processed. The final distances represent the shortest travel time from the ambulance's source location to every intersection.

5. Find the Nearest Hospital:

After calculating the shortest distances, the distances corresponding to all hospital locations are compared. The hospital with the minimum distance is selected as the nearest hospital in terms of travel time.

Example

Consider an ambulance starting at intersection **S** with three possible hospitals located at different nodes. Each road has a weight representing its travel time in minutes.

Suppose the calculated shortest travel times are:

- Hospital H1 → **12 minutes**
- Hospital H2 → **8 minutes**
- Hospital H3 → **15 minutes**

The algorithm compares these values and selects **H2**, since it has the minimum travel time. Thus:

Nearest Hospital = H2

Shortest Travel Time = 8 minutes

Dynamic Traffic Conditions

In a smart traffic management system, road weights may change because of congestion, accidents, or other traffic conditions. When the travel time of a road changes, its edge weight can be updated in the graph. Dijkstra's algorithm can then be executed again using the updated weights to determine the new shortest routes.

For example, if a road initially requires **5 minutes** but congestion increases its travel time to **12 minutes**, its edge weight can be updated before recalculating the shortest path.

Complexity Analysis

Using an adjacency list and a priority queue, Dijkstra's algorithm efficiently handles large road networks.

Time Complexity = O((V + E) log V)

where:

- **V** = number of intersections (vertices)
- **E** = number of roads (edges)

For a sparse graph, this is commonly expressed as:

O(E log V)

The adjacency list and distance array require:

Space Complexity = O(V + E)

Pseudo Code

```
START
Input intersections, roads and travel times
Create graph
Input ambulance location and hospital locations
Run Dijkstra from ambulance location
Find hospital with minimum travel time
Display nearest hospital and optimal route
WHILE traffic update is required
 Input changed road and new travel time
 Update road weight
 Run Dijkstra again
 Find new nearest hospital
```

Display updated route and travel time

END WHILE

STOP

Advantages

● Finds the shortest path efficiently for graphs with non-negative edge weights. ● Suitable for navigation and emergency vehicle routing.

● Can find shortest distances from one source to multiple destinations.

● Priority queues improve efficiency for large road networks.

● Road weights can be updated to represent changing traffic conditions.

Limitation

Dijkstra's algorithm does not work correctly when the graph contains negative edge weights. Also, if traffic conditions change frequently, repeatedly running the algorithm may introduce additional computational overhead.

Hence, Dijkstra's Algorithm is an effective approach for smart traffic management and emergency vehicle navigation, as it can determine the minimum travel-time route and help ambulances reach suitable hospitals as quickly as possible.

GitHub Repository:

https://github.com/Harshit000-byte/DAA-ASSIGNMENTS-124B1F017-HARSHIT-PANDITA.git

Conclusion:

The smart traffic management system uses Dijkstra's algorithm to provide ambulances with the fastest route to hospitals, dynamically updating paths based on real-time traffic conditions. Scalable for large city networks and supported with visual navigation, it ensures minimal response time, improved emergency handling, and contributes to building safer, smarter cities.